NIDS: Final Viva Guide (Everything, From Zero)

This single file teaches the whole project: what it is, how every part works, how to run and demonstrate it, and the questions the panel is likely to ask with answers in your own words. Read sections 1 to 4 first, then practise the demo, then read the rest.

The project is also saved as a report, a presentation and the source code: - Report: `report/NIDS_Mini_Project_Report.pdf` (57 pages) - Presentation: `NIDS_Final_Presentation_Rohan_Pandey.pptx` (10 slides) - Logbook: `report/Weekly_Progress_Logbook.pdf` - Code online: https://github.com/rohanpandey436/realtime-nids

---

1. The project in one minute

"My project is a real-time Network Intrusion Detection System for small networks like a home, hostel or lab. A Python program captures every packet on my laptop's Wi-Fi and loopback using Scapy. A rule engine I wrote checks each packet for three attacks: port scans, SYN floods and ARP spoofing. When a rule fires, an alert is sent as JSON to a Node.js server, which logs it and pushes it to a web dashboard over Socket.IO, so a red banner appears within about a second. I use plain rules instead of machine learning, so every alert can be traced to a counter and a threshold and I can explain it line by line."

The whole system is about 200 lines of code and runs on an ordinary laptop.

---

2. Background concepts you must know

- **IDS vs IPS vs firewall.** A firewall decides what traffic is allowed. An IPS blocks what it detects. An IDS only watches and raises an alert. Mine is an IDS, so it is safe to run live.
- **NIDS vs HIDS.** A network IDS watches packets on the wire; a host IDS watches one machine's logs. Mine is a NIDS.

- **Signature vs anomaly detection.** Signature = match known patterns or thresholds. Anomaly = model normal behaviour and flag deviations. My threshold rules are a form of signature detection.
- **Packet and layers.** A frame on the wire has an Ethernet header, then an IP header, then a TCP or UDP header, then the data. ARP sits directly on Ethernet and has no IP header. Scapy gives each layer as an object whose fields I can read.
- **TCP three-way handshake.** Client sends SYN, server replies SYN-ACK, client sends ACK. Flags are bits: SYN opens, ACK acknowledges, RST resets, FIN closes.
- **Half-open connection.** A SYN the server answered but never got the final ACK for. A flood of these fills the server's backlog so real users are refused.
- **ARP.** Maps an IP address to a MAC (hardware) address on the local network. It has no authentication and hosts accept replies they never asked for, which is what spoofing abuses.
- **Sliding window.** Counting only the events of the last few seconds; old events are dropped as time moves, so the count is always "in the last N seconds".
- **Port scan / SYN flood / ARP spoofing.** Reconnaissance (which ports are open), denial of service (exhaust the server), and man-in-the-middle (redirect traffic through the attacker).

---

3. Architecture and the journey of one packet

```
Wi-Fi / loopback --Npcap--> Scapy sniff --> handle() --
> rules
     rules fire --> outbox queue --> sender thread --
HTTP POST /alert--> server.js
     server.js --> append to alerts.log  +  Socket.IO
"alert" --> dashboard (banner + row)
     every 1 s: stats thread --POST /stats--> server --
Socket.IO "stats"--> live chart
```

Four components: 1. **Sniffer** (`sensor/nids.py`): captures and decodes packets. 2. **Detection rules** (same file): three functions that keep sliding-window counters. 3. **Server** (`server/server.js`): receives, validates, logs and forwards alerts. 4. **Dashboard** (`server/public/index.html`): shows live counts, a chart and alerts.

---

4. How to run the demo (three terminals)

```
# terminal 1 - server
cd C:\Users\pande\work\nids-project\server
```

```
node server.js

# terminal 2 - sensor
cd C:\Users\pande\work\nids-project\sensor
python nids.py

# terminal 3 - attacks
cd C:\Users\pande\work\nids-project\sensor
python scan_test.py     # port scan  -> orange (medium)
alert
python flood_test.py    # SYN flood  -> red (high)
alert
```

Open http://localhost:3000. Put the browser beside the terminals.

- The counters move as soon as the sensor runs; open a website to make them jump.
- `scan_test.py` raises an orange **Port Scan** alert within a second.
- `flood_test.py` raises a red **SYN Flood** alert.
- Show the saved log: `type ..\server\alerts.log`.
- Run a test again within 30 seconds: no new alert. That is the cool-down, not a bug.

ARP spoofing cannot be shown on one laptop, because the loopback adapter does not carry ARP frames. Say this honestly: it is proven by a unit test, and a live demo would need a second device.

Why the alerts say 127.0.0.1: the test attacks come from your own laptop, and localhost is 127.0.0.1. The sensor listens on Wi-Fi and loopback, so a self-test works on one machine.

---

5. The three detection rules (with the real code)

All rules live in `sensor/nids.py` and share a helper and an alert function.

```
def trim(history, now):              # drop events
older than the window
    while history and now - history[0][0] > WINDOW:
        history.popleft()

def alert(kind, severity, src, dst, detail, packet,
subject):
    now = float(packet.time)
    if now - last_alert.get((kind, subject), 0) <
COOLDOWN:   # cool-down
        return
    last_alert[(kind, subject)] = now
    record = {"type": kind, "severity": severity,
"src": src, "dst": dst,
```

```
              "detail": detail, "time":
time.strftime("%H:%M:%S", time.localtime(now))}
    outbox.put(("/alert", record))   # hand to the
sender thread
```

5.1 Port scan (medium)

For each source IP, keep a deque of (time, port) for lone SYN packets. Count distinct ports in the last 5 seconds; alert above 15.

```
if tcp.flags == "S":
    history = syn_by_src[ip.src]
    history.append((now, tcp.dport)); trim(history,
now)
    ports = {port for _, port in history}
    if len(ports) > cfg["port_scan_ports"]:
        alert("Port Scan", "medium", ip.src, ip.dst,
              "%d ports in %d s" % (len(ports),
WINDOW), packet, ip.src)
```

Why only SYN: a scanner sends one SYN to each port. Normal browsing uses a few ports, so it never reaches 15 on one host. Cool-down key = source (one scanner, one alert).

5.2 SYN flood (high)

Per destination, count SYNs and completed handshakes (ACK set, SYN clear) in the window. Alert when many SYNs, few ports, and almost no completions.

```
syns = syn_by_dst[ip.dst]; syns.append((now,
tcp.dport)); trim(syns, now)
trim(ack_by_dst[ip.dst], now); completed =
len(ack_by_dst[ip.dst])
if len(syns) >= cfg["syn_flood_syns"] and len({p for _,
p in syns}) <= 3 \
        and completed < len(syns) / 10:
    alert("SYN Flood", "high", ip.src, ip.dst,
          "%d SYN, %d completed" % (len(syns),
completed), packet, ip.dst)
```

Why: a real burst completes handshakes; a flood does not, because sources are forged. Cool-down key = **destination** (the victim), because a real flood comes from many forged sources at once; keying on the source would give one alert per source, which is the flood of alerts we want to avoid. (This was found during unit testing and fixed with the `subject` argument.)

5.3 ARP spoofing (critical)

At start-up, learn the gateway's MAC into a trust table. Alert on any ARP reply that claims a trusted IP with a different MAC.

```
def check_arp(packet):
    arp = packet[ARP]
    known = trusted.get(arp.psrc)
    if arp.op == 2 and known and arp.hwsrc.lower() !=
known.lower():
        alert("ARP Spoofing", "critical", arp.psrc,
arp.pdst,
              "MAC changed from %s to %s" % (known,
arp.hwsrc), packet, arp.psrc)
```

Why: the attacker keeps announcing the gateway's IP at their own MAC to sit in the middle. Watching for a changed MAC on a known IP is the arpwatch principle.

---

6. The rest of the sensor

- **config.json**: all thresholds, the interface name and the gateway live here, so nothing is hard-coded. Loaded once at start-up.
- **Three threads**: the main thread captures and runs rules; a stats thread reports counts every second; a sender thread does the HTTP POSTs. They are joined by one thread-safe queue (`outbox`). This keeps the capture path fast, so no packet is missed while the network is slow. If the server is down, the sender catches the error and capture continues.
- **The filter**: `"(tcp or udp or arp) and not port 3000"` runs inside Npcap, so Python only sees the packets the rules need and never its own dashboard traffic.
- **store=False**: captured packets are not kept, so memory stays constant.

7. The server (`server/server.js`, Node.js + Express + Socket.IO)

- `POST /stats`: forward the per-second counts to all browsers.
- `POST /alert`: reject anything missing type/src/severity with 400; else add the date, append one JSON line to `alerts.log`, and emit to browsers.
- `GET /api/alerts`: return the last 100 alerts, which the page loads so history survives a refresh.

8. The dashboard (`server/public/index.html`)

- Connects with Socket.IO, shows a status line, three protocol counters, a Chart.js line graph, a colour-coded banner (orange/red/purple) and an alert table with severity pills and per-type totals.

- Uses `textContent` (not innerHTML) for all network data, so a crafted address cannot inject a script. This is a deliberate security choice (cross-site scripting prevention).

---

9. Testing (what to say)

- **10 unit tests** (`tests/test_rules.py`), all pass. They build packets with Scapy, give them timestamps, feed them to `handle()`, and check the alerts. Half are **negative** tests: replies are not a scan, slow activity is ignored, completed handshakes are not a flood, a scan is not a flood, genuine ARP is silent, the cool-down blocks repeats. Run: `python -m unittest tests.test_rules -v`.
- **Integration tests**: real port scan and SYN flood against the host; the alert reached the dashboard and log in about 2 ms; no false alarm during normal browsing.
- **Why negative tests matter**: a detector that cries wolf gets ignored, so staying silent on normal traffic is as important as catching attacks.

10. Results (the numbers)

Metric	Result
Attacks detected	3 (port scan, SYN flood, ARP spoofing)
Unit tests	10, all passing
Detection latency	about 2 ms (one Npcap loopback outlier of a few seconds, not on real Wi-Fi)
Packets during flood test	several hundred per second, no loss
False alarms in normal browsing	none observed
Lines of code	about 200

---

11. The report (so you can navigate it in the viva)

57 pages. Front matter (cover, certificate, declaration, acknowledgement, abstract, contents, lists). Then: Ch1 Introduction, Ch2 Literature Review (Wireshark/Snort/Zeek, the attack papers, why rules not ML), Ch3 System Analysis and Design (requirements, DFDs, use-case, sequence, component, deployment, data design), Ch4 Implementation (every module with code excerpts), Ch5 Testing, Ch6 Results, Ch7 Conclusion and Future Scope, References (IEEE), and an Appendix with the full source code. Figures are numbered chapter-wise (Fig. 3.1 etc.); so are tables.

12. Questions the panel may ask (answers)

- **What does your project do?** Captures live packets, detects three attacks with transparent rules, and shows alerts on a web dashboard within about a second.
- **IDS vs firewall?** A firewall blocks by policy; my IDS only watches and alerts.
- **Why rules, not machine learning?** Rules are explainable, need no training data, and run in real time. I can trace every alert to a counter and a threshold.
- **How does the port-scan rule work?** Count distinct destination ports from one source in a 5-second sliding window; alert above 15, then a 30-second cool-down.
- **Why only SYN packets?** A scanner sends one SYN per port; replies come from the server.
- **How do you tell a flood from a busy server?** A flood has many SYNs but almost no completed handshakes; a busy server completes them.
- **Why is the flood cool-down keyed on the victim?** A real flood uses many forged sources; keying on the source would make one alert per source.
- **How do you detect ARP spoofing?** Learn the gateway's MAC at start-up; alert if a reply claims the gateway's IP with a different MAC.
- **What is Scapy / Npcap?** Npcap is the Windows driver that captures raw packets; Scapy is the Python library that decodes them into layers.
- **Why Python and Node together?** Python is best for capture; Node with Socket.IO is simplest for live browser updates. They talk over HTTP with JSON, so each is testable alone.
- **Why Socket.IO not refresh?** It keeps one connection open and pushes alerts instantly.
- **What if the server is off?** The sender catches the error and capture keeps running.
- **What can you see on Wi-Fi?** Your own host's traffic and broadcasts; full LAN coverage needs a mirror port. That is why tests target the laptop itself.
- **Can an attacker evade it?** Yes: a very slow scan or one spread over many sources can stay under the thresholds. Longer windows and per-destination counting would help; it is future work.
- **Is scanning legal?** Only on your own devices or with permission. My tests target 127.0.0.1.
- **Database?** No database yet; recent state is in memory and alerts are a JSON log. SQLite is future work.
- **What is left / future scope?** More rules (ICMP, DNS, SSH), notifications, SQLite storage, sensor authentication, and an active-response mode that firewalls the attacker.

13. Live changes they might ask for

- Alert at 11 ports: `config.json`, set `port_scan_ports` to 10, restart the sensor.
- Longer window: set `window_seconds` to 10.
- Shorter cool-down: set `cooldown_seconds` to 5, scan twice to get two alerts.
- Keep the banner longer: in `index.html`, change `6000` to `10000`, refresh the page.

14. Before the viva

- Run the full demo once in the morning and take two screenshots (banner + terminal) as a backup.
- Carry the printed report, the logbook (signed), and the presentation.
- Be ready to open any file and explain any function.
- Run the plagiarism check through the institution's tool and attach the report.